Final Year Project Implementation Master Guide

AI-Powered Adaptive Study Group Matching Platform

From zero implementation to a professional mid-demo-ready system

Purpose	A full beginner-friendly roadmap to build your project properly, explain each technical part, and prepare a strong product-based mid demo.
Target Outcome	Working frontend screens, connected backend API, database integration, matching logic, basic AI/ML story, and clean documentation/screenshots.
Recommended Stack	React + Tailwind CSS frontend, FastAPI backend, PostgreSQL database, optional scikit-learn for later ML experiments.
Best Strategy	Build the core product first with rule-based matching. Add lightweight ML only after the main workflow is working.

Prepared as a practical guide for implementation, demo completion, and Chapter 5 writing.

Contents

- 1. What your mid demo expects
- 2. Recommended project scope for full marks
- 3. Technology stack and why each part is used
- 4. Proper system architecture
- 5. Professional project folder structure
- 6. Step-by-step implementation path
- 7. Database design and tables
- 8. Backend API design and coding order
- 9. Frontend screen flow and coding order
- 10. Matching logic, AI, ML, and model training explained simply
- 11. External API integration explained
- 12. Testing, screenshots, and demo evidence
- 13. 14-day and 7-day execution plans
- 14. Demo presentation flow
- 15. Chapter 5 documentation guidance
- 16. Copy-paste AI prompts for every phase
- 17. Final checklist

1. What your mid demo expects

For a product-based project, examiners usually look for evidence that the system is real, usable, connected, and understandable. They do not only want slides or diagrams. They want to see that the key workflow of the product works from the user interface through the backend and database.

For your project, the strongest demo is not a very advanced AI model on day one. The strongest demo is a complete workflow: a student registers, enters preferences, gets match recommendations, joins a study group or buddy match, sees scheduled availability, and uses a basic chat or message area.

That means your first target should be product completeness and usability. A simple but fully connected implementation scores better than a partially built project with many unfinished advanced ideas.

What you should show in the mid demo

Area	What examiners want to see	What you should build
Frontend	Usable screens, consistent navigation, clear user flow	Login, register, dashboard, preference form, match results, group page, admin page
Backend	Real logic, proper routes, input/output handling	FastAPI routes for auth, users, preferences, matches, groups, messages
Database	Stored data linked to features	PostgreSQL tables for users, preferences, groups, memberships, messages, schedules
AI/ML story	A believable intelligent component	Start with rule-based similarity matching; later add ML or recommendation improvements
Integration	Frontend, backend, database work together	React calls backend APIs; backend reads/writes database
Evidence	Proof of implementation	Screenshots, API tests, database tables, running demo, short explanation

2. Recommended project scope for full marks

Your project title suggests a smart platform with social preferences and subject-based matching. To keep the scope realistic and professional, divide the system into core features and optional enhancements.

Core modules to finish first

- User Management Module: registration, login, profile update, role handling.
- Preference Module: subjects, availability, study style, communication preference, preferred group type.
- Matching Module: recommend suitable partners/groups using a similarity score.
- Study Group Module: create group, join group, see members, leave group.
- Scheduling Module: store available days and times, view compatible slots.
- Messaging Module: basic one-to-one or group chat/log messages.
- Admin Module: view users, groups, reported issues, basic analytics.

Optional enhancements after the core workflow is stable

- Notification emails or in-app notifications
- Calendar integration
- Advanced ML recommendation model
- Sentiment analysis or engagement scoring
- Video meeting links

3. Technology stack and why each part is used

Layer	Recommended technology	Why it fits this project	Priority
Frontend	React + Vite + Tailwind CSS	Fast to build modern screens and easy to demo	High
Backend	FastAPI + Uvicorn	Clean API structure, fast development, automatic docs	High
Database	PostgreSQL	Reliable relational database for user/group data	High
ORM	SQLAlchemy	Maps Python objects to database tables	High
Authentication	JWT	Simple token-based login flow	High
Testing	Postman / FastAPI Swagger / pytest	Easy API testing and proof	Medium
ML later	scikit-learn	Good for simple recommendations and experiments	Medium
Deployment later	Render / Railway / Vercel / EC2	Useful after local system is stable	Low for mid demo

Important decision

Do not start with a complicated machine learning pipeline. For the mid demo, use a strong rule-based matching engine first. That lets you show intelligent recommendations without being blocked by model training. Later, you can say that the system is designed to evolve into an ML-driven recommender as more interaction data becomes available.

4. Proper system architecture

Use a clean client-server architecture. The frontend handles screens and user interaction. The backend handles business logic and API endpoints. The database stores all permanent system data. The matching engine sits in the backend layer, because it needs access to user preferences and compatibility rules.

- Client layer: React user interface
- Application layer: FastAPI routes, services, matching logic
- Data layer: PostgreSQL database
- Optional intelligence layer: recommendation utilities, scoring functions, ML experiments

Suggested request flow

1. 1. User opens frontend screen.
2. 2. Frontend sends request to backend API.
3. 3. Backend validates request and checks authentication.

4. 4. Backend service logic processes the request.
5. 5. Database is read or updated.
6. 6. Backend returns JSON response.
7. 7. Frontend displays the result to the user.

5. Professional project folder structure

Use one main project folder and separate the frontend, backend, documentation, and model/experiment areas. This keeps your work organized and makes it much easier to explain during the demo.

```
study-group-matching-platform/
├── README.md
├── .gitignore
├── docs/
│   ├── proposal/
│   ├── diagrams/
│   │   ├── use-case/
│   │   ├── activity/
│   │   ├── sequence/
│   │   ├── class/
│   │   ├── erd/
│   │   └── architecture/
│   ├── screenshots/
│   │   ├── frontend/
│   │   ├── backend/
│   │   ├── database/
│   │   └── demo-flow/
│   ├── chapter-5/
│   └── test-evidence/
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── api/
│   │   ├── assets/
│   │   ├── components/
│   │   │   ├── common/
│   │   │   ├── layout/
│   │   │   └── forms/
│   │   ├── features/
│   │   │   ├── auth/
│   │   │   ├── profile/
│   │   │   ├── preferences/
│   │   │   ├── matching/
│   │   │   ├── groups/
```

```
│   │   │   ├── scheduling/
│   │   │   ├── chat/
│   │   │   └── admin/
│   │   ├── hooks/
│   │   ├── pages/
│   │   ├── routes/
│   │   ├── store/
│   │   ├── utils/
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── package.json
│   └── vite.config.js
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   ├── security.py
│   │   │   └── database.py
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── routes/
│   │   ├── services/
│   │   ├── repositories/
│   │   ├── matching/
│   │   │   ├── scoring.py
│   │   │   ├── rules.py
│   │   │   └── recommender.py
│   │   ├── utils/
│   │   └── tests/
│   ├── requirements.txt
│   └── .env
├── ml/
│   ├── notebooks/
│   ├── datasets/
│   ├── training/
│   ├── saved_models/
│   └── evaluation/
├── scripts/
│   ├── seed_data.py
│   ├── run_backend.bat
│   ├── run_frontend.bat
│   └── export_demo_data.py
└── deployment/
```

```
├── docker/
├── nginx/
└── cloud-notes/
```

What each main folder means

Folder	Purpose
docs	All academic deliverables: proposal, diagrams, screenshots, Chapter 5 content, and evidence.
frontend	React user interface code.
backend	FastAPI application, routes, schemas, models, business logic.
ml	Optional experiment space for training or evaluating recommendation models.
scripts	Quick utilities for seeding test data and running the system.
deployment	Hosting and deployment notes for later stages.

6. Step-by-step implementation path

Step 1: Finalize the minimum feature list

Freeze the scope first. Decide exactly what must work for the demo: registration, login, preferences, matching, group creation, schedule view, and basic chat/log messages.

Step 2: Set up the repository and folder structure

Create the project root, frontend, backend, docs, and ml folders exactly once. Do not start coding inside random folders.

Step 3: Design the database

Identify entities and relationships before writing API code. This reduces rework.

Step 4: Build the backend foundation

Set up FastAPI, PostgreSQL connection, SQLAlchemy models, and migrations or table creation.

Step 5: Build authentication

Implement register, login, password hashing, and JWT token handling.

Step 6: Build profile and preference APIs

Allow users to save subjects, availability, study style, communication preference, and preferred group size.

Step 7: Build the matching logic

Start with a rule-based compatibility score. This is your first intelligent feature.

Step 8: Build study group APIs

Create, list, join, and leave groups.

Step 9: Build scheduling APIs

Store and retrieve available time slots.

Step 10: Build messaging or communication log

A simple chat table and send/list endpoints are enough for a good demo.

Step 11: Build frontend pages in the same order

Connect forms one by one to the backend.

Step 12: Add seed data and test data

Demo quality improves a lot when your database already contains realistic sample users.

Step 13: Capture screenshots and evidence

After each working feature, save screenshots and API response proof.

Step 14: Prepare demo script and Chapter 5 write-up

Explain what each feature does, which files implement it, and how the database supports it.

7. Database design and tables

A clean relational design is essential because your project depends on well-structured student preferences and group relationships.

Table	Important fields	Why it exists
users	id, full_name, email, password_hash, role, created_at	Stores all users and admins
profiles	user_id, faculty, year, bio, skill_level	Stores additional student info
preferences	user_id, preferred_subjects, study_style, communication_mode, preferred_group_type	Stores matching preferences
availability	user_id, day_of_week, start_time, end_time	Stores time slots
study_groups	id, name, subject, created_by, max_members	Stores created groups
group_memberships	group_id, user_id, status, joined_at	Tracks membership
messages	id, sender_id, receiver_id or group_id, content, sent_at	Stores chat or communication logs
matches	id, user_id, matched_user_id, score, explanation	Stores recommended matches
notifications	user_id, title, body, is_read	Optional notification support

Recommended ERD relationships

- One user has one profile.

- One user has one preference record or multiple normalized preference records.
- One user can have many availability slots.
- One user can create many study groups.
- One study group can have many members.
- One user can send many messages.
- One user can have many match recommendations.

8. Backend API design and coding order

Build the backend in vertical slices. Finish one feature end-to-end before moving to the next.

Module	Example routes
Auth	POST /auth/register, POST /auth/login, GET /auth/me
Profile	GET /profile/me, PUT /profile/me
Preferences	GET /preferences/me, POST /preferences, PUT /preferences/me
Availability	POST /availability, GET /availability/me, DELETE /availability/{id}
Matches	GET /matches/recommendations, POST /matches/generate
Groups	POST /groups, GET /groups, GET /groups/{id}, POST /groups/{id}/join, POST /groups/{id}/leave
Messages	POST /messages, GET /messages/group/{id}, GET /messages/user/{id}
Admin	GET /admin/users, GET /admin/groups, GET /admin/stats

Recommended backend order

8. database.py and config.py
9. User model, schema, and auth routes
10. Profile and preference routes
11. Availability routes
12. Matching service and routes
13. Group routes
14. Message routes
15. Admin routes

9. Frontend screen flow and coding order

Screen category	Pages
Public	Landing page, login, register
User setup	Complete profile, add subjects, select study style, set availability
Main usage	Dashboard, recommended matches, suggested groups
Interaction	Group details, join group, view members,

	message area
Management	Edit preferences, edit profile, notifications
Admin	Admin dashboard, user list, group overview

Recommended frontend build sequence

16. Create layout, navbar, route protection, and auth context first.
17. Build login and register forms.
18. Build dashboard shell and placeholder cards.
19. Build preference form and connect it to backend.
20. Build match result cards and detail view.
21. Build group pages.
22. Build schedule/availability components.
23. Build chat or message area.
24. Add admin pages last.

10. Matching logic, AI, ML, and model training explained simply

This is the part that usually feels confusing. The easiest professional approach is to build it in three maturity levels.

Level 1: Rule-based matching (best starting point)

Calculate compatibility using predefined rules. Example: same subject = +30, overlapping availability = +25, same study style = +20, same communication preference = +10, similar skill level = +15. This is simple, explainable, and excellent for a first demo.

Level 2: Weighted scoring with feedback

Keep the same scoring system but adjust the weights when users accept or reject recommendations. This makes the system feel adaptive.

Level 3: ML recommendation model

After you collect enough interaction data, you can train a model that predicts whether two users are likely to be a good match.

What model training means in your project

Model training means teaching a machine learning algorithm to learn patterns from historical data. In your case, the historical data might include user preferences, availability overlap, past accepted matches, rejected matches, group participation, or chat engagement. The model would learn which combinations of features usually produce good study matches.

Why you should not start with model training immediately

- You currently have no large labeled dataset.
- You need the product to work first.
- A simple rule-based system is easier to explain and defend in the demo.

- You can still describe the system as AI-enabled by using intelligent matching logic and showing how it can evolve into ML.

Simple ML path for later

25. Collect sample records of user A + user B + match outcome.
26. Create features such as subject similarity, time overlap, study style match, communication match, skill gap, previous join rate.
27. Train a classification or ranking model using scikit-learn.
28. Evaluate using accuracy, precision, recall, or ranking quality.
29. Save the model and load it in the backend recommendation service.

Pseudo-code for rule-based matching

```
score = 0
if same_subject: score += 30
if availability_overlap: score += 25
if same_study_style: score += 20
if same_communication_pref: score += 10
if skill_level_close: score += 15
return score
```

11. External API integration explained

An API integration means your system communicates with another external service. For this project, API integration is optional for the mid demo. Do not force external APIs unless they clearly improve the workflow.

Possible integration	Use case	Should you do it now?
Google Calendar API	Sync study sessions or create calendar events	Later
Email API	Send match or group notifications	Optional
Video meeting API	Generate meeting links	Later
Firebase or socket service	Real-time chat	Optional later
No external API	Use internal database-based features only	Best for current stage

Recommended decision for your mid demo

Skip heavy external integrations for now. Instead, complete internal APIs between your React frontend and FastAPI backend. This is the integration that matters most for marks because it proves your product is working.

12. Testing, screenshots, and demo evidence

Every completed feature should generate evidence. Save this inside docs/screenshots and docs/test-evidence.

- Screenshot of the frontend page
- Screenshot of the Swagger or Postman API test
- Screenshot of the database table or inserted record
- Short note explaining what the feature does

Minimum evidence set for the mid demo

- Register page working
- Login page working
- Preference submission working
- Availability saving working
- Match recommendation shown
- Group create/join flow working
- Message send/list flow working or at least communication log shown
- Admin view working
- Database records visible

13. Execution plans

14-day implementation plan

Day range	Focus	Expected output
Day 1-2	Folder structure, environment setup, database design	Repository ready, ERD draft, project skeleton
Day 3-4	Backend auth + user/profile tables	Register/login APIs working
Day 5-6	Preference and availability APIs	Preference and availability saved to DB
Day 7-8	Matching logic and match endpoints	Recommendation results returned
Day 9-10	Group and membership APIs	Create/join/list groups working
Day 11	Message feature or communication log	Basic messaging flow working
Day 12	Frontend integration and UI polishing	Connected user flow
Day 13	Testing, screenshots, bug fixes	Evidence set prepared
Day 14	Presentation script + Chapter 5 notes	Demo-ready package

Emergency 7-day compressed plan

30. Day 1: setup, database, auth
31. Day 2: profile + preferences
32. Day 3: availability + matching
33. Day 4: groups
34. Day 5: frontend integration
35. Day 6: screenshots, test data, polish
36. Day 7: demo rehearsal and write-up

14. Demo presentation flow

37. Start with the problem: students struggle to find suitable study partners.
38. State the solution: an AI-powered adaptive study group matching platform.
39. Show the architecture in one slide.
40. Demonstrate the user journey live: register, login, enter preferences, generate matches, join group, view schedule, send message.
41. Show the database briefly.
42. Explain the matching logic clearly and confidently.
43. End with future improvements: feedback learning, advanced ML, calendar integration.

15. Chapter 5 documentation guidance

Your Chapter 5 should explain what you built, not only what you planned. A strong structure is shown below.

44. 5.1 System Navigation and Module Structure
45. 5.2 Development Environment
46. 5.3 Tools and Technologies
47. 5.4 Database Implementation
48. 5.5 Backend Implementation
49. 5.6 Frontend Implementation
50. 5.7 Matching Logic / AI Component
51. 5.8 Testing and Results
52. 5.9 Summary

Under each section, add screenshots, short explanations, and references to important files or routes.

16. Copy-paste AI prompts for every phase

Prompt 1 - Project structure

> I am building a final year project called AI-Powered Adaptive Study Group Matching Platform. I am using React for the frontend, FastAPI for the backend, and PostgreSQL for the database. Give me a clean professional full-stack folder structure for this project with explanations for every main folder and subfolder. Keep it realistic for a student project and suitable for a mid demo.

Prompt 2 - Database design

> For an AI-powered study group matching platform, design a PostgreSQL database schema with tables for users, profiles, preferences, availability, study groups, group memberships, messages, matches, and notifications. Explain primary keys, foreign keys, and relationships in simple words.

Prompt 3 - Backend roadmap

> I am a beginner. Give me a step-by-step FastAPI backend implementation roadmap for a study group matching platform. Start from environment setup, database connection, auth, user profile, preferences, availability, matching service, groups, and messaging. For each step, explain what files to create and why.

Prompt 4 - Frontend roadmap

Give me a React + Tailwind CSS implementation roadmap for a study group matching platform. Include page order, component structure, route protection, API integration, state management, and a recommended file structure. I want a clean professional UI suitable for a university project demo.

Prompt 5 - Matching logic

Help me design a rule-based matching engine for a study group matching platform. The matching should consider subject interests, availability overlap, communication preference, study style, and skill level. Give me the scoring logic, pseudo-code, backend service design, and sample output.

Prompt 6 - AI/ML explanation

Explain how AI and machine learning can be used in a study group matching platform for a final year project. Start with a simple rule-based approach and then explain how I can later collect data and train a scikit-learn model. Keep it professional but easy for a beginner to understand.

Prompt 7 - Chapter 5 writing

Generate Chapter 5 System Implementation for my final year project called AI-Powered Adaptive Study Group Matching Platform. Include module descriptions, development environment, database implementation, backend implementation, frontend implementation, matching logic, testing, and screenshot placeholders. Use academic writing style.

Prompt 8 - Mid demo checklist

Create a mid-demo readiness checklist for a product-based university project called AI-Powered Adaptive Study Group Matching Platform. The checklist should cover working features, screenshots, API proof, database proof, UI polish, and presentation preparation.

17. Final checklist

☐ Folder structure created properly

☐ Frontend and backend separated cleanly

☐ Database schema finalized

☐ Register/login implemented

☐ Preferences and availability stored

☐ Matching endpoint working

☐ Group creation and membership flow working

☐ Basic messaging or communication log working

☐ Frontend connected to APIs

☐ Seed data prepared for demo

☐ Screenshots saved in docs folder

☐ Chapter 5 notes started

☐ Presentation flow rehearsed

Final advice

Your best path to full marks is not to chase complexity too early. Build a complete, clean, connected system first. Make the product usable. Make the code structure professional. Make the documentation easy to understand. Then position the AI and ML component as an intelligent matching engine that starts with rules and grows into data-driven recommendations over time.